Executive Summary

Isaac Assink, Brendan Malaugh, Eric Zeng

April 2026

1 Introduction

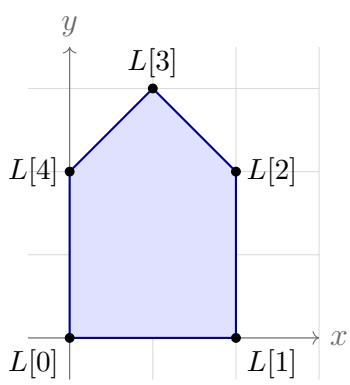
Our project explores the Minkowski sum, which is a way of adding together two sets by taking the set of all pairwise sums. More precisely, for any set S on which addition is defined, we have the Minkowski sum of $A, B \subseteq S$ as

$$A \oplus B = \{a + b : a \in A,\ b \in B\}$$

For computing Minkowski sums over infinite sets, we first need to establish a representation, because storing them as literal sets is impossible. A common case of this is when $S = \mathbb{R}^2$ and A and B are polygons. We store a polygon A as a list of vertices V_A of length n where there is an edge between points $V_A[i]$ and $V_A[(i+1) \mod n]$ for each $0 \leq i \leq n-1$. The polygon that L represents is then the region bounded by the union of those edges. For example, the vertex list

$$V_A = \{(0,0),\ (2,0),\ (2,2),\ (1,3),\ (0,2)\}$$

corresponds to the polygon A, shown below.



For arbitrary polygons A and B, it turns out that the Minkowski sum can take a very long time; in the worst case, its $O(|V_A|^2 \cdot |V_B|^2)$ time. However, when both polygons are convex, we can actually achieve guaranteed $O(|V_A| + |V_B|)$ performance. As a reminder, a convex polygon is one in which the line segment between any two points in the polygon lies entirely in the polygon, or equivalently if all interior angles are $\leq 180°$ (along with no self-intersection). The figure above is an example of a convex polygon. The case of the Minkowski sum of two convex polygons is what will be the main focus of this paper.

This problem is difficult enough that even a "naive" solution is somewhat hard to find, but there is decently simple (though slow) approach that uses only what we already know. The idea is to do a Minkowski sum on the list representations of A and B, then take the convex hull of the resultant set. That is,

$$A \oplus B = \text{HULL}(V_A \oplus V_B)$$

The Minkowski sum step takes $O(|V_A| \cdot |V_B|)$ time, and $V_A \oplus V_B$ contains up to $|V_A| \cdot |V_B|$ points, so the convex hull step takes $O(|V_A| \cdot |V_B| \cdot \log(|V_A| \cdot |V_B|))$ time, which is also the overall time complexity. This is way slower than are target complexity, so we somehow need a way to obtain this hull without having to exhaustively compute everything in the Minkowski sum.

2 Solution

The key fact we will be using to solve this problem is that if A and B are convex polygons, then the set of edges of $A \oplus B$ is exactly the union of the set of edges of A and B, up to translation. That is, there exists some way to translate each of the edges in A and B to construct the polygon $A \oplus B$. We will also be using the fact that convexity of A and B implies convexity of $A \oplus B$, which is useful since it immediate forces the ordering of the edges in $A \oplus B$. This is because a convex polygon must have edges which always make counter-clockwise turns, which implies the edges must come in order of polar angle away from some fixed direction.

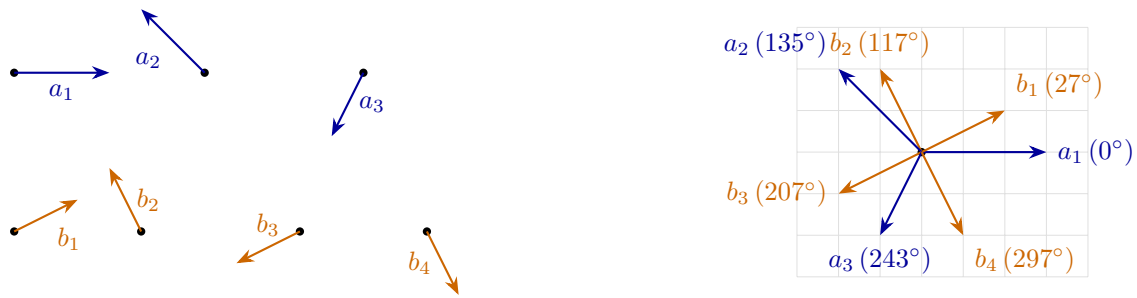
For the purposes of the algorithm and the demonstration below, the we will choose our reference direction to be the positive x-axis. We represent each polygon as a counter-clockwise cycle of vectors stacked tip-to-tail, encoding the edges and orientation of the polygon.

(a) Polygon A

(b) Polygon B

Figure 1: The two input polygons with their edges as CCW vectors.

We now detach the directed edges from each polygon, consider them as one big set of vectors, and sort them by polar angle away from the positive x-axis.
Lastly, we recombine the vectors by stacking them tip-to-tail, starting with the one that has the smallest polar angle and moving counterclockwise (from smallest to largest polar angle). This gives us the resultant polygon up to translation. To figure out its absolute position in space, we just need to figure out where to anchor the first vector. This can be done by finding the bottom-most point of A and B (with ties broken by taking the left-most point) and adding them together. The sum of those two points is the bottom-most point of $A \oplus B$, and is the starting point of the first vector in the construction. In this case, the bottom-most point of both A and B is $(0, 0)$, so the first vector in the construction of $A \oplus B$ is anchored at the origin.



(a) Edges detached from both polygons

(b) Gathered at origin, sorted CCW

Figure 2: Detach all edge vectors, then sort them by angle.

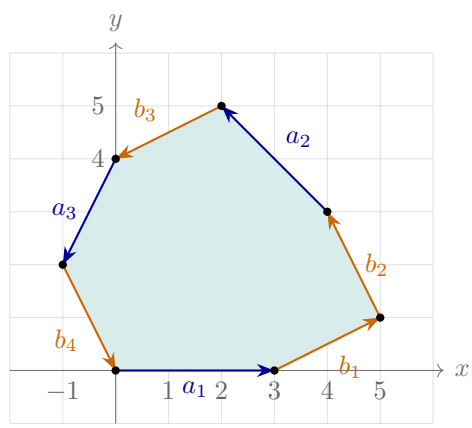


Figure 3: Reassembled tip-to-tail in sorted order: $A \oplus B$.

3 Implementation

The input to Minkowski sum is two polygons. Each polygon can be modeled as a list of points sorted counterclockwise. The first step to applying the algorithm is sorting the edges by by polar angle. This can be done by shifting this point list such that the first element is the lowest y value, and the lowest x value if there are several points with the same minimum y value. The first edge will then have the smallest polar angle, and since the points are sorted counterclockwise, each subsequent edge will have a higher polar angle than the last.
Then, we make these polygon lists cyclic (for ease of implementation) by appending the first and second points to the end of each polygon, and create pointers for each polygon starting at the first element. Each pointer will correspond with the edge with point at the pointer and the point after the pointer.

While we still have unprocessed edges, we will add the edge that has a lower polar angle. If both pointers' corresponding edges are equal, then we will add the sum of both. In the code, the cross product between the edges determine which edge's polar angle is larger (relatively). The two cyclic points ensure the edge never goes out of bounds. The result algorithm performs the Minkowski Sum as described in section 2.

```python
def minkowskiSum(poly1, poly2):
    poly1 = reorderPolygon(poly1)
    poly2 = reorderPolygon(poly2)
```

(a) The input to Minkowski is two polygons, modeled as a CCW list of points

```python
def reorderPolygon(polygon):
    minVertexIdx = 0
    for i in range(len(polygon)):
        if (polygon[i].y < polygon[minVertexIdx].y
            or (polygon[i].y == polygon[minVertexIdx].y
                and polygon[i].x < polygon[minVertexIdx].x)):
            minVertexIdx = i
    return polygon[minVertexIdx:] + polygon[:minVertexIdx]
```

(b) Reordering a polygon so edges are sorted by polar angle.

Figure 4: Python snippets of Minkowski Sum algorithm

```python
# two points loop around for simpler implementation
# makes sure each edge is accounted for
poly1.append(poly1[0])
poly1.append(poly1[1])
poly2.append(poly2[0])
poly2.append(poly2[1])

# pointers for iterating through the polygons
ptr1 = 0
ptr2 = 0

polySum = []
while (ptr1 < len(poly1)-2 or ptr2 < len(poly2)-2):
    # append sum of current points
    polySum.append(poly1[ptr1] + poly2[ptr2])

    # compute cross product, determines which edge is more ccw
    cross = (poly1[ptr1+1] - poly1[ptr1]).cross(poly2[ptr2+1] - poly2[ptr2])
    if cross > 0:
        ptr1 += 1
    elif cross < 0:
        ptr2 += 1
    else: # if equally ccw increment both
        ptr1 += 1
        ptr2 += 1

return polySum
```

Figure 5: Python snippet of main loop of Minkowski Sum algorithm

4 Programming Challenge

Removing the flavor text, the programming challenge asks the following.

You are given n disjoint, convex polygons. You can build "bridges" between polygons (that is, from one point on a polygon to a point on another polygon), where the cost of a bridge is its length. The goal is then to make some number of bridges such that for any two islands, there is some path to reach one from the other. Then, output the minimal total cost (that is, the sum of the cost across all bridges). To pass, the problem needs to be solved in $O(n^2m)$ time, where m is the number of points in each polygon.

5 Solving the Programming Challenge

This programming challenge is hard, and perhaps most comparable to the Salesmole homework assignment. First, it is expected that the solver understands the Minkowski sum, and also understands that it can be used to efficiently find the distance between two polygons. This was covered in our presentation, but we explain the argument again here.

We wish to find the minimum distance between two polygons A and B. Let us think of these as infinite sets of points for the moment. Now, consider their minimum distance d. There are thus some two points $a \in A$, $b \in B$ such that $d = |a - b|$. Let us then consider the minkowski sum $A + (-B)$. This is exactly the set $\{a - b : a \in A, b \in B\}$. If the origin is in this set, then $a = b$ for some $a \in A$, $b \in B$, so the minimum distance is zero. Otherwise, we notice that the minimum of $|x|$ over $x \in A + (-B)$ is exactly the distance between the origin and $A + (-B)$. We can thus compute the distance between A and B by taking the Minkowski sum of A and $-B$, checking if the origin is contained in the result. If it is not, we then compute the distance from the origin to each vertex of $A + (-B)$ and to each line segment. The minimum across all of these is the desired distance. Since the Minkowski sum of two convex polygons has at most as many vertices as the sum of the two initial polygons, this is linear in the number of vertices.

Similar to the Salesmole homework assignment, we now wish to construct a graph with each island as a vertex, and edges being the distance between the corresponding islands. After constructing this graph, we then take the minimum spanning tree, and output the cost of it. This will give a set of bridges with the minimum possible cost.

Thus, the only remaining part is constructing the graph. For each pair of islands, we can compute the distance between them in $O(m)$ time. Thus, computing it for all pairs can be done in $O(n^2m)$ time. The immediate issue, however, is that a bridge could go through another polygon. There is also not enough time to check this. However, if a bridge were to go through some middle polygon, then in the MST, it would be better to go to that middle polygon, and then go to the last polygon. This means that we are able to naively include these edges in our graph, understanding that they will not be included in the resulting MST. Computing the MST is $O(n)$ time, so we thus return the answer in $O(n^2m)$ time.

6 Conclusion

Minkowski Sum is an elegant solution to a problem that sounds niche but has several applications. For convex polygons, Minkowski's algorithm shortens the runtime greatly over the naive solution. If both polygons have n vertices, the runtime improves from $\Theta(n^2 \log(n))$ to $\Theta(n)$. The implementation is also exceptionally simple. It follows a two pointer approach, merging the two lists of edges.

There are only a few special cases to consider (namely when two edges point in the same direction).

The most important application to know is that a Minkowski difference can calculate the minimum distance between polygons. This is the main concept behind our programming challenge, and also can be extended to robotics and collision detection as well (see our slides!).